Samrey Repository Bug Report

Complete findings, rounds 1 and 2
github.com/vis6ky/Samrey | Reviewed October 27, 2026

How this review was done

I cloned the repository, installed dependencies, ran the typecheck and the repo's own test suite, and wrote small probe scripts (using jsdom for DOM behavior) for each finding. Items are **reproduced** unless tagged *(read)*, which means the problem was found by reading the source but not executed. XSS items were confirmed by parsing the sanitizer output in a DOM, not by running in a real browser.

Headline results

- `tsc --noEmit` passes, but `npm test` runs **278 tests and 4 fail** (all in `tests/phase24/`). The README badge says 130 passed, 0 failed.
- The hand-written `el()` / `bind*` runtime path mostly works. The repo's own docs app booted in jsdom and survived 54 button clicks with no errors.
- The compiler, the JSX/Vite plugin, the CLI commands, SSR hydration, DevTools wiring and several security claims do not work as documented.
- No secrets are committed, and every starter-template import resolves to a real export.

Contents

- **Part 1** - Round 1: features not implemented, security, reactivity and store, runtime, router/HTTP/forms/SSR, tooling and repo
- **Part 2** - Round 2: starter project and Vite plugin, DevTools, tests and release gates, docs versus code, benchmarks, metadata, things that checked out
- **Part 3** - Earlier README and package.json observations, coverage gaps, suggested next steps

Part 1 - Round 1 findings

1.1 Features that don't work or aren't implemented

- **CLI commands are stubs.** In an empty directory:
  - `build` writes nothing but prints "Build finished" and lists `dist/client.js`.
  - `test` runs no tests and always exits 0 ("All test suites passed").
  - `analyze` prints hard-coded numbers (14.2 KB, 18 signals).
  - `preview` starts no server.
  - `audit` scans two hard-coded sample strings, reports "Audit passed", and exits 0 unless `--strict`. Its checker misses `javascript:` hrefs and inline `onclick`; it only checks `<img>` without `alt=` and the literal `<script`.
- **The compiler emits wrong code.**
  - `bindText(root.childNodes[slotIndex])` ignores the computed DOM path. `<h2>Count: {n}</h2>` renders `0` because the whole `<h2>` is replaced.
  - Dynamic attributes and events bind to the root element, so a click anywhere in the component fires the handler.
  - `<Show>`, `<For>` and components compile to HTML comments only.
  - `{'Hello ' + name + '!'}` is folded into the constant text `Hello ' + name + '!`. This hits any expression that starts and ends with a quote or backtick.
  - `{null}{undefined}{false}` renders `nullundefinedfalse`.
  - Whitespace between inline elements is dropped (`<b>a</b> <i>b</i>` becomes "ab").
  - A literal `<` in text is dropped.
  - `<style>.a { color: red }</style>` turns the CSS braces into an interpolation and the CSS is lost.
  - Static `onclick="..."`, `srcdoc` and `<object data="javascript:...">` pass through to the output.
  - Broken templates still generate code, and the `options` argument is ignored.
  - `lintA11y` and the security classifier's output are never used by the compile path.
- **The README Quick Start crashes.**
  - `bindEvent` is not exported (nor are `bindAttribute` and `bindProperty`, which the docs list).
  - `template()` returns only `clone.firstChild`. For the README's own indented template that is a whitespace text node with no `querySelectorAll`. Multi-root templates silently drop every node after the first.
- **No real hydration.**
  - `ssr.hydrate` always calls `replaceChildren` with fresh client nodes.
  - Its mismatch check only compares `nodeType`, so `<button>` and `<section>` match.
  - Replayed events go to old SSR nodes that were just removed, so pre-hydration clicks are lost.
  - `runtime.hydrate` is just `mount`.
- **Stubs and dead code.**
  - `onError()` is an empty function.
  - `Suspense` is a try/catch.
  - `RequestCache` (deduplication) is never used by `createResource` or `HttpClient`, and `createHttpCache` ignores its options.
  - The CHANGELOG claims LIS reconciliation, but `For` does not do it.

1.2 Security

- `sanitizeHtmlString` **bypasses.** Event-handler attributes survive in `<svg/onload=...>` and `<img src="x"/onerror=...>`. `href` values survive as `javascript:` (unquoted, or `javascript:`) or as `data:text/html;base64`. Tab-split schemes such as `java	script:` also pass, relying on URL parsers stripping tabs. `rawHtml` and the Trusted Types policy both use this sanitizer.
- **Static attributes skip sanitization in** `el()`**.** `el('a', {href: 'javascript:...'})`, `{onclick: '...'}` (a string) and `{srcdoc: ...}` are set raw. Only function-valued attributes go through `bindAttr`.
- `bindAttr` **with** `null` **yields** `href=""` **and** `src=""`**.** An empty `href` reloads the current page.
- **Open redirect.** `sanitizeRedirectUri('/\t/evil.com')` passes, and browsers strip the tab to get `//evil.com`.

- **SSR head injection.** `addMeta` values and `clientScript` / `clientStyles` are not escaped; a `</script>` injection worked.
- **SSG path traversal.** A param of `..` wrote to `/tmp/index.html`, outside `outDir`, and `/blog/..` overwrites the root `index.html`. Params that are prefixes of other params also collide (`:id` inside `:idx`).
- **Dev-server traversal.** `GET /../secret.txt` served a file outside the root. The server also listened on a non-loopback interface even with the default host `localhost`.
- `stripSecrets` **(two copies):**
  - Shared references are silently dropped (`{a: u, b: u}` loses `b`).
  - `Date` and `Map` become `{}`.
  - It strips `author`, `authorId`, `hashtags`, `sessions` and `tokenCount`.
  - It misses `passwd`, `pwd`, `accessKey` and `ssn`.
- `/api/assistant` **in** `vite.config.ts` *(read)*. It is unauthenticated, reads an unbounded body, and lets the caller inject the system prompt via `retrievedDocs`. The `dev` script binds `0.0.0.0`, so anyone on the network can spend the owner's Gemini quota.
- **Trusted Types:**
  - `createScriptURL` allows any `https:` origin.
  - The runtime never calls the policy *(read, no usages found)*, so `rawHtml` would throw under `require-trusted-types-for`.
- **DevTools bridge** *(read)*. It installs on import (production too) and broadcasts every event with `postMessage(..., '*')`. `isOriginAllowed` is never called.
- **CSRF tokens** *(read)*. They fall back to `Math.random()` when the global `crypto` is missing, which Node 18 lacks while the README claims to support it. A string "secret" argument is silently ignored.
- **The inline event-delegator** `<script>` **has no nonce option**, which conflicts with the "CSP strict" badge.

1.3 Reactivity and store

- `Owner.dispose` skips every other child (four children disposed `[0, 2]`).
- One throwing effect kills its siblings and its own future runs.
- Disposed-while-queued effects resurrect.
- Nested effects accumulate on re-run (3 runs instead of 1).
- `computed` has no equality cutoff, so effects re-run on unchanged values.
- Writes made during an effect's own run are lost (`s.value++` inside an effect stopped at 1).
- A signal holding a function treats the new value as an updater.
- `createStore`:
  - Array mutations are not reactive.
  - `Date` and `Map` throw `TypeError` (the proxy is the receiver).
  - `delete` and `Object.keys` are not tracked.

1.4 Runtime

- `Show` and `For` do not untrack. Any signal read in a child re-renders the whole branch.
- `Show` re-mounts when a truthy value changes identity.
- `For` never passes updated items to existing rows. Same key plus new data leaves the old UI.
- `For` and `Show` mount in a microtask. An update before the flush leaves **ghost rows** (got `a,b,c` instead of `c`).
- `Portal` never cleans up on dispose.
- `bindDynamicChild` does not reorder retained nodes (`[A,B]` to `[B,A]` stays `AB`).
- `Dynamic` leaks a `children="..."` attribute.
- Headless `el()` renders function children as source text and does not escape text in `outerHTML`, which is the SSR path *(read)*.

- Dev diagnostics are dead in browsers because `isDev` depends on `process`. The messages still carry `AETH-` codes left over from a rename.

1.5 Router, HTTP, Forms, SSR

- **Router:**
  - The initial path ignores `window.location`.
  - Back and Forward do not update the router (after Back, window was `/a` and router was `/b`).
  - Matching fails with `?query` or `#hash` (and `/users/:id` captures `5?tab=2`).
  - `navigate('about')` and `navigate('?x')` are silent no-ops.
  - `Link` returns a plain object, not a Node. It rewrites external `href`s to `/` and ignores `activeClass`.
  - `createLink` swallows external link clicks.
  - A module-level router singleton leaks between concurrent SSR requests *(read)*.
  - `useSearchParams` never syncs with the URL.
- **HTTP:**
  - `createResource` never re-fetches when its source signal changes.
  - Out-of-order responses win (a stale "a" overwrote "b").
  - `timeoutMs` is ignored.
  - A `Headers` instance in options drops all custom headers.
  - `DELETE` returning 204 throws on `.json()`.
  - `FormData` is sent as `{}` with a JSON content-type.
- **Forms:**
  - Double submit runs `onSubmit` twice.
  - Schema paths like `[{key: 'email'}]` (the Standard Schema object form) become a form-level error.
  - Schema-set field errors never clear when the user fixes the value.
  - `dirty` is true for equal-but-new arrays.
- **SSR:**
  - `createServerData` runs every loader twice per request.
  - The second-pass render shows `loading=true` ("LOADING..." in the HTML despite state).
  - A failing second-pass loader causes `unhandledRejection`, which crashes Node by default.
  - `renderToStream` loses `<title>` and meta tags and does not actually stream; its `abortSignal` and `timeoutMs` options are ignored.
  - The client data cache never invalidates.

1.6 Tooling, testing, repo

- **CI cannot run.** Only `bun.lock` is tracked, so `npm ci` and `setup-node` with `cache: npm` fail. CI never tests Node 18, and `lint` duplicates `typecheck`.
- **Metadata and docs:**
  - Package versions are 1.0.0 but the README says 0.1.0.
  - `repository` and `homepage` point to `samrey-framework/samrey` and `samrey.dev`, not this repo.
  - Packages ship raw `.ts` as `main`.
  - The root `package.json` has leftover AI Studio dependencies.
- **CLI:**
  - `samrey create --template x` creates a directory named `--template`.
  - `create .` overwrites existing files, and `generate` overwrites without checking.
  - `samrey generate component my-widget` emits invalid identifiers (`My-widgetProps`).
  - Scaffolds use JSX with `"jsx": "react-jsx"` but there is no React or Samrey JSX runtime.
- **Testing package:**
  - `fireEvent` sends a plain `Event`, so `e.button` is undefined and `Link` / `createLink` handlers silently do nothing in tests (a real `MouseEvent` navigated).
  - `cleanupAllTestContainers` targets a marker nothing creates.

  - `createMockResource` is not reactive and does not match the real API.
- **The 4 failing tests** are all in `tests/phase24/` (assistant UI and hybrid doc search), so main is red.

Part 2 - Round 2 findings

2.1 Biggest finding: `samrey create` projects can't build

- `samreyVitePlugin` **replaces your TypeScript with a template string.** It runs every `.tsx`, `.jsx` and `.samrey` file through the HTML-template compiler. The starter's `App.tsx` came out as `template("import <!-- dynamic-slot --> from '@samrey/router';...")`, with no real `App` export.
- **The plugin ignores its** `exclude` **option** and also transforms `.tsx` files inside `node_modules`.
- **The starter's router renders nothing useful.** `<Router><Route/></Router>` hands `Route` props objects, and `mount()` turned them into the text `[object Object]`.
- **The starter's scripts are stubs.** `npm run build`, `test`, `audit` and `analyze` call the fake CLI commands from round 1.

2.2 DevTools

- **Nothing feeds it.** Outside `runtime/diagnostics.ts`, no package calls the bridge or any inspector. The dev banner's "DevTools: Enabled" is cosmetic.
- **Secrets leak through URLs.** `NetworkInspector` redacts headers but emitted `?access_token=SECRET123` verbatim.
- **Memory grows without bound.** After 5,000 requests it retained 5,001, while the bridge log is capped at 200. The owner, effect and graph registries never evict either.
- **Any page script can read the broadcast**, including form field values. A plain `message` listener heard all 5,002 events.
- `sanitizePayload` **shares the flaws of** `stripSecrets`**:**
  - Shared (non-circular) references become `[Circular Reference]`.
  - `author` is redacted as if it were a secret.
  - Tokens embedded in longer strings, such as `use Bearer abc123 now`, are not redacted.

2.3 Tests and release gates don't test the claims

- **The hydration example is fake.** It imports `hydrate` but never calls it, hard-codes `{success: true, replayedEventsCount: 0}`, and `tests/examples.test.ts` asserts that constant.
- **The "adversarial XSS" test uses only textbook payloads** (`<script>`, `<img onerror>`, `javascript:`). It never asserts that `onerror` is removed, and none of the round 1 bypasses are covered.
- **The "0 orphaned subscriptions" lifecycle test** uses a fake container and a single flat effect per mount, so it cannot see the owner-disposal or nested-effect leaks.
- **The "public APIs match specification" tests** only compare export lists.
- **Docs quote different test counts.** They say 130 or 236, the suite actually runs 278, and 4 fail.

2.4 Docs reference APIs that don't exist

Checked by importing every `@samrey/*` package and comparing against named imports in 125 markdown files. These names are imported in docs but not exported:

Missing name	Package	Where documented
`bindEvent`	runtime	README
`samreyPlugin`	cli	packages/cli/README.md
`initDevtools`, `getGraphSnapshot`	devtools	packages/devtools/README.md
`bindField`	forms	docs/FORMS_AND_VALIDATION.md
`validators`	forms	packages/forms/README.md
`RouterOutlet`	router	packages/router/README.md
`useNavigate`	router	docs/DEVELOPER_ASSISTANT_BLIND_BENCHMARK.md

Missing name	Package	Where documented
`generatePreHydrationScript`	ssr	examples/hydration/README.md
`serializeState`	ssr	packages/ssr/README.md
`mount`	testing	docs/TESTING_AND_TOOLING.md

`SECURITY.md` also claims compatibility with strict `script-src 'self'` and compile-time attribute escaping. The SSR inline event-delegator script (no nonce option) and the unsanitized static attributes contradict both.

2.5 Benchmarks *(read, gate not run)*

- The committed baseline was recorded on a 2-CPU sandbox (`cpuModel: "unknown"`), while the thresholds are absolute ops/sec floors. That is machine-dependent, so the CI gate will be flaky or meaningless.
- The "Full Compile with Show/For" benchmarks measure a compiler that emits nothing for `<Show>` and `<For>`.

2.6 Provenance and metadata

- Version strings disagree four ways: README says 0.1.0, packages say 1.0.0, and `SECURITY.md` and several docs say `1.0.0-rc.1`.
- The repo URL appears as `samrey-framework/samrey` (23 times), `samrey-dev/samrey` (2 times), and the actual `vis6ky/Samrey`.
- `CITATION.cff` has placeholder authors and an all-zero ORCID.
- `docs/research/SAMREY_RESEARCH_PAPER.md` is generated by a script from hard-coded strings and also has placeholder authors. Its numbers have not been verified.

2.7 Things that checked out

- `tsc --noEmit` is clean, and no secrets are committed.
- Every starter-template import resolves to a real export.
- `SECURITY.md`'s secret-stripping examples (`password`, `apiKey`, `jwtToken`, `databaseUrl`, `secret`) are all stripped.
- The repo's own docs/task-studio app boots in jsdom and survived 54 button clicks with no errors.

Taken together, the framework mostly works when used through the hand-written `el()` and `bind*` API. The compiler, the JSX/Vite path, the CLI and the SSR hydration layer are where it breaks down.

Part 3 - Observations, gaps and next steps

3.1 Early README and package.json observations

- The README's Node 18 claim conflicts with the `test` script: `node --import tsx --test` needs Node 18.18+ or 20.6+, and Node 18 is past end-of-life.
- The `test` script relies on shell glob expansion, which breaks in Windows `cmd`, and mixes named files with globs, so new test files in some packages are silently skipped. `tests/developer-assistant` is not in the list.
- The "130 passed / 0 failed" badge is a hard-coded image, not tied to CI.
- The README Hello World selects spans by index with `querySelectorAll('span')`, which breaks when the template changes.
- The root `package.json` lists `express`, `@google/genai`, `dotenv` and `vite` as runtime dependencies, plus unrelated `pdfkit` and `docx` dev dependencies.
- `clean` uses `rm -rf` (not Windows-portable), and `dev` binds `--host=0.0.0.0`.

3.2 Coverage gaps

Not reviewed in depth: prose in `docs/`, the benchmark code itself, parts of `playground/` and `app/applet`, and most of the logic inside `src/task-studio` (it was exercised by booting and clicking, not read). The benchmark numbers and the research paper's claims were not verified.

3.3 Suggested next steps

- Patch the reactivity core first (owner disposal, effect error handling, nested effect cleanup, computed equality), since much else depends on it.
- Replace the regex HTML sanitizer, and sanitize static attributes in `el()`.
- Fix the compiler's slot-path binding and constant folding, or stop shipping the Vite plugin until it handles real `.tsx`.
- Make the CLI commands do real work, or label them as placeholders.
- Add tests for the bypass payloads, nested-owner disposal and a real hydration round trip.
- Fix CI by committing a `package-lock.json` (or switching CI to Bun), and repair the 4 failing tests.